SVN版本控制

1.SVN介绍

1.1 SVN简介

- SVN全称Subversion，是一个开放源代码的版本控制系统，管理着随时间改变的代码。这些代码放置在一个中央资料档案库(repository) 中。 这个档案库很像一个普通的文件服务器,不过它会记住每一次文件的变动。 这样你就可以把档案恢复到旧的版本, 或是浏览文件的变动历史。说得简单一点SVN就是用于多个人共同开发同一个项目，管理共用资源的目的。

1.2 SVN主要作用

- 目录版本控制
  Subversion 实现了一个 "虚拟" 的版本控管文件系统, 能够依时间跟踪整个目录的变动。目录和文件都能进行版本控制。
- 真实的版本历史
  Subversion中，可以对文件和目录进行增加（add）、删除（delete）、复制（copy）和重命名（rename）等操作。所有的新加的文件都从一个新的、干净的版本开始。
- 自动提交
  一个提交动作，不是全部更新到了档案库中，就是完全不更新。这允许开发人员以逻辑区间建立并提交变动，以防止当部分提交成功时出现的问题。

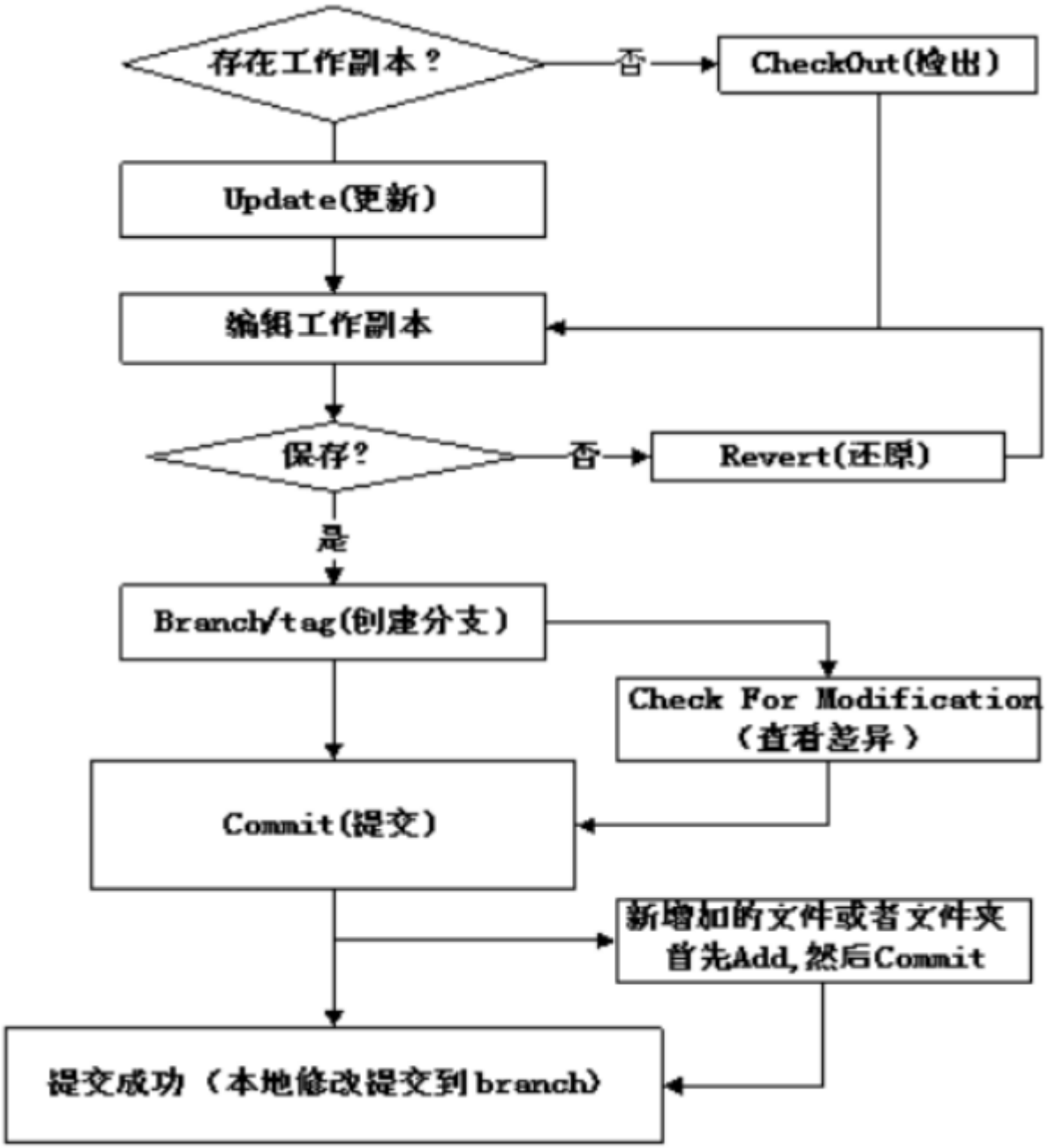
1.3 SVN基本操作

- Repository（源代码库）：源代码统一存放的地方
- Checkout（提取）：当你手上没有源代码的时候，你需要从repository checkout一份
- Commit（提交）：当你已经修改了代码，你就需要Commit到repository
- Update (更新)：当你已经Checkout了一份源代码， Update后就可以和Repository上的源代码同步

1.4 SVN工作流程

1. 如果本地没有代码，从服务器下载项目组最新代码。（Checkout）
2. 如果已经Checkout并且有人已Commit了代码，你可以更新以获得最新代码。（Update）
3. 进入自己的分支，进行工作，每隔一个小时向服务器自己的分支提交一次代码（很多人都有这个习惯。因为有时候自己对代码改来改去，最后又想还原到前一个小时的版本，或者看看前一个小时自己修改了哪些代码，就需要这样做了）。（Commit）
4. 下班时间到了，把自己的分支合并到服务器主分支上，一天的工作完成，并反映给服务器。（Commit）

PS：如果两个程序员同时修改了同一个文件， SVN可以合并这两个程序员的改动，实际上SVN管理源代码是以行为单位的，就是说两个程序员只要不是修改了同一行程序，SVN都会自动合并两种修改。如果是同一行，SVN会提示文件Confict, 冲突，需要手动确认。



1.5 SVN生命周期

1.5.1 创建版本库

- 版本库相当于一个集中的空间，用于存放开发者所有的工作成果。版本库不仅能存放文件，还包括了每次修改的历史，即每个文件的变动历史。
- Create 操作是用来创建一个新的版本库。大多数情况下这个操作只会执行一次。当你创建一个新的版本库的时候，你的版本控制系统会让你提供一些信息来标识版本库，例如创建的位置和版本库的名字。

1.5.2 检出

- Checkout 操作是用来从版本库创建一个工作副本(从远程仓库拉取到本地)。工作副本是开发者私人的工作空间，可以进行内容的修改，然后提交到版本库中。

1.5.3 更新

- 顾名思义，update 操作是用来更新版本库的。这个操作将工作副本与版本库进行同步。由于版本库是由整个团队共用的，当其他人提交了他们的改动之后，你的工作副本就会过期。所以你必须通过update操作从远程仓库获取最新版本的代码

1.5.4 执行变更

- 当你从远程仓库获取到代码后可以对文件和目录进行新增、删除、修改等操作，这些变动只是对本地项目进行操作，只有在commit操作后，才会提交到远程仓库。

1.5.5 复查变化

- 当你检出工作副本或者更新工作副本后，你的工作副本就跟版本库完全同步了。但是当你对工作副本进行一些修改之后，你的工作副本会比版本库要新。在 commit 操作之前复查下你的修改是一个很好的习惯。

  Status 操作列出了工作副本中所进行的变动。正如我们之前提到的，你对工作副本的任何改动都会成为待变更列表的一部分。Status 操作就是用来查看这个待变更列表。

  Status 操作只是提供了一个变动列表，但并不提供变动的详细信息。你可以用 diff 操作来查看这些变动的详细信息。

1.5.6 修复错误(版本回滚)

- 我们来假设你对工作副本做了许多修改，但是现在你不想要这些修改了，这时候 revert 操作将会帮助你。Revert操作重置了对工作副本的修改。它可以重置一个或多个文件/目录。当然它也可以重置整个工作副本。在这种情况下，revert操作将会销毁待变更列表并将工作副本恢复到原始状态。

1.5.7 版本冲突

- 合并的时候可能会发生冲突。Merge 操作会自动处理可以安全合并的东西。其它的会被当做冲突。例如，"hello.c" 文件在一个分支上被修改，在另一个分支上被删除了。这种情况就需要人为处理。Resolve 操作就是用来帮助用户找出冲突并告诉版本库如何处理这些冲突。

  PS:当两人修改同一行代码并提交到远程仓库（或者更新）时才会出现冲突

1.5.8 提交修改

- Commit 操作是用来将更改从工作副本到版本库。这个操作会修改版本库的内容，其它开发者可以通过更新他们的工作副本来查看这些修改。

  PS:在提交之前，你必须将文件/目录添加到待变更列表(版本控制管理中)中。列表中记录了将会被提交的改动。当提交的时候，我们通常会提供一个注释来说明为什么会进行这些改动。这个注释也会成为版本库历史记录的一部分。Commit是一个原子操作，也就是说要么完全提交成功，要么失败回滚。用户不会看到成功提交一半的情况。

2. SVN安装

2.1 SVN下载

- SVN服务器下载 [服务器下载链接]
- SVN客户端下载 [客户端下载链接]

3.SVN操作

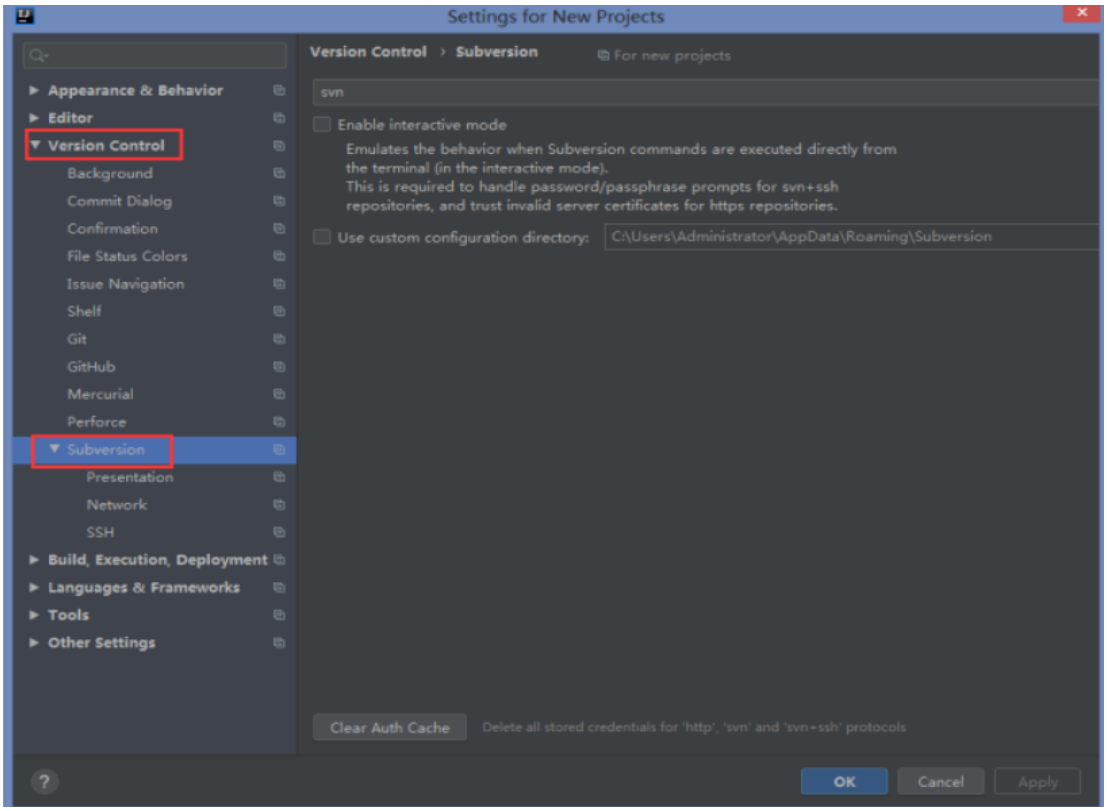
- SVN原生操作 [SVN原生操作]
- TortoiseSVN操作 [TortoiseSVN操作]

PS:素材目录中有SVN笔记

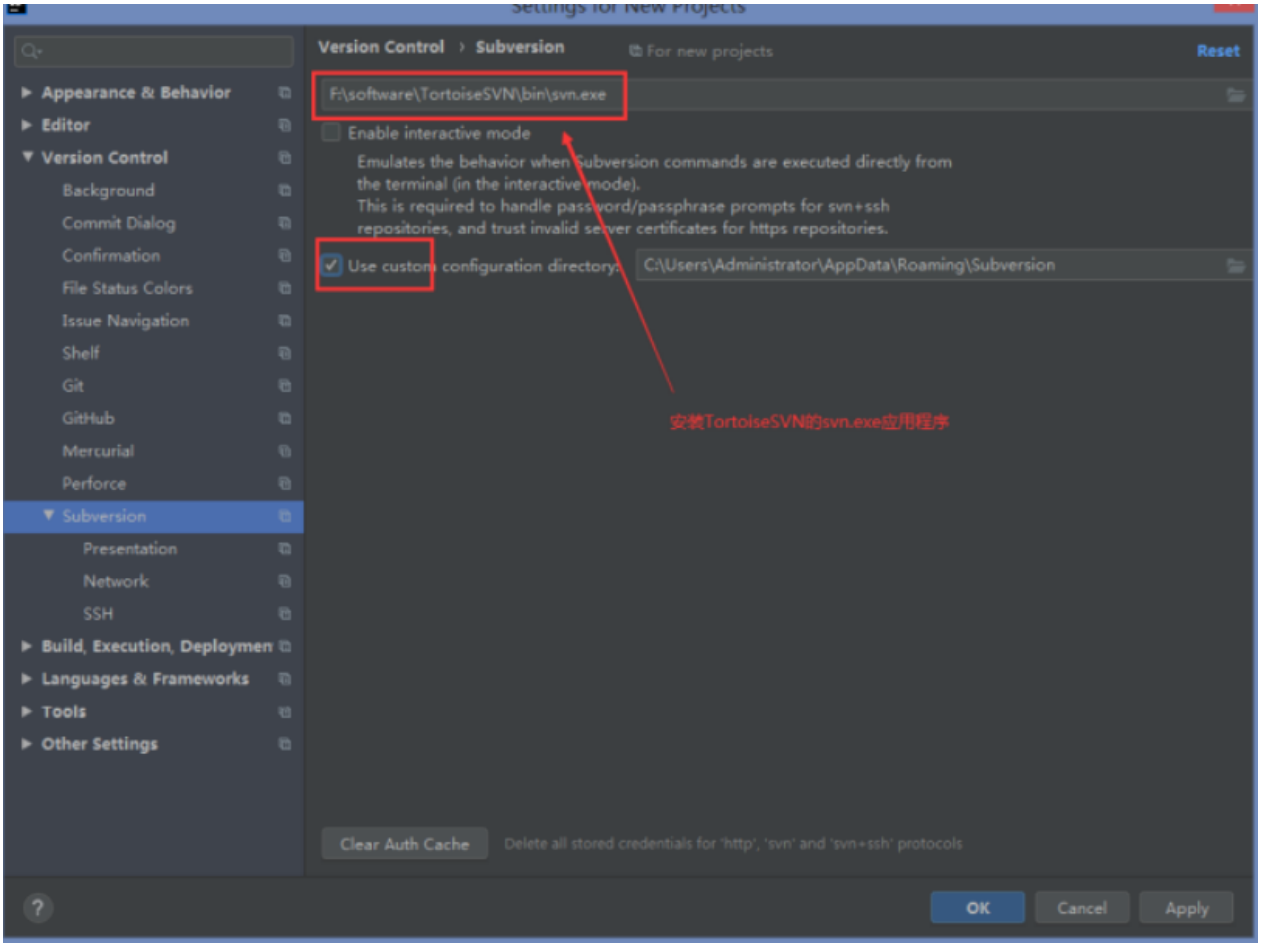
4.IDEA集成SVN

4.1 IDEA配置SVN

1. File —> Other Settings（全局配置；Settings是局部配置） —> Version Control —> Subversion



2. 配置svn

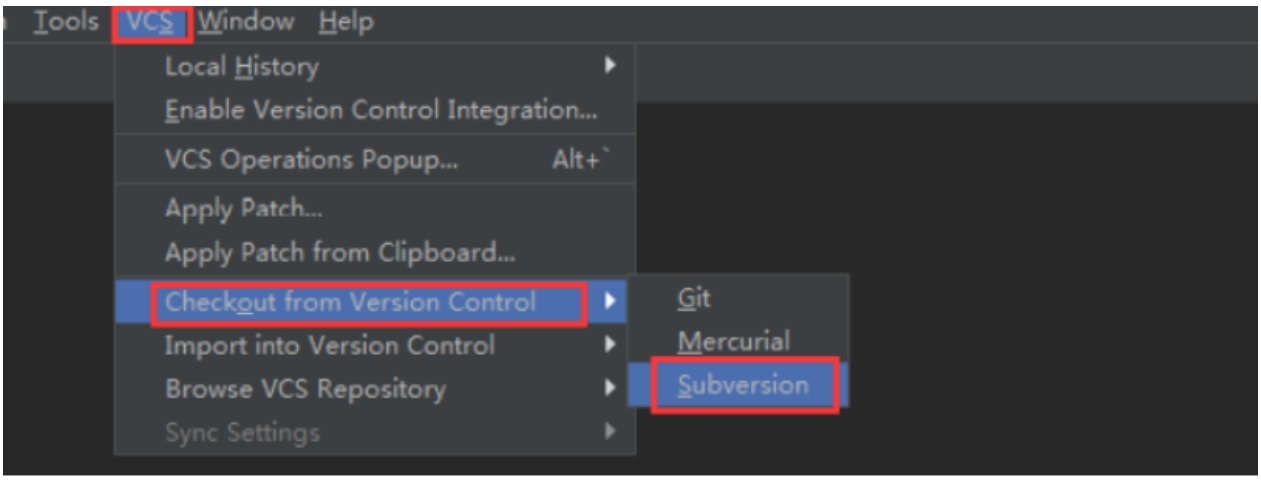


PS:找不到svn.exe文件，TortoiseSVN的bin目录下面没有svn.exe
之所以没有是因为安装TortoiseSVN的时候没有勾选指定安装项，添加command line client tools

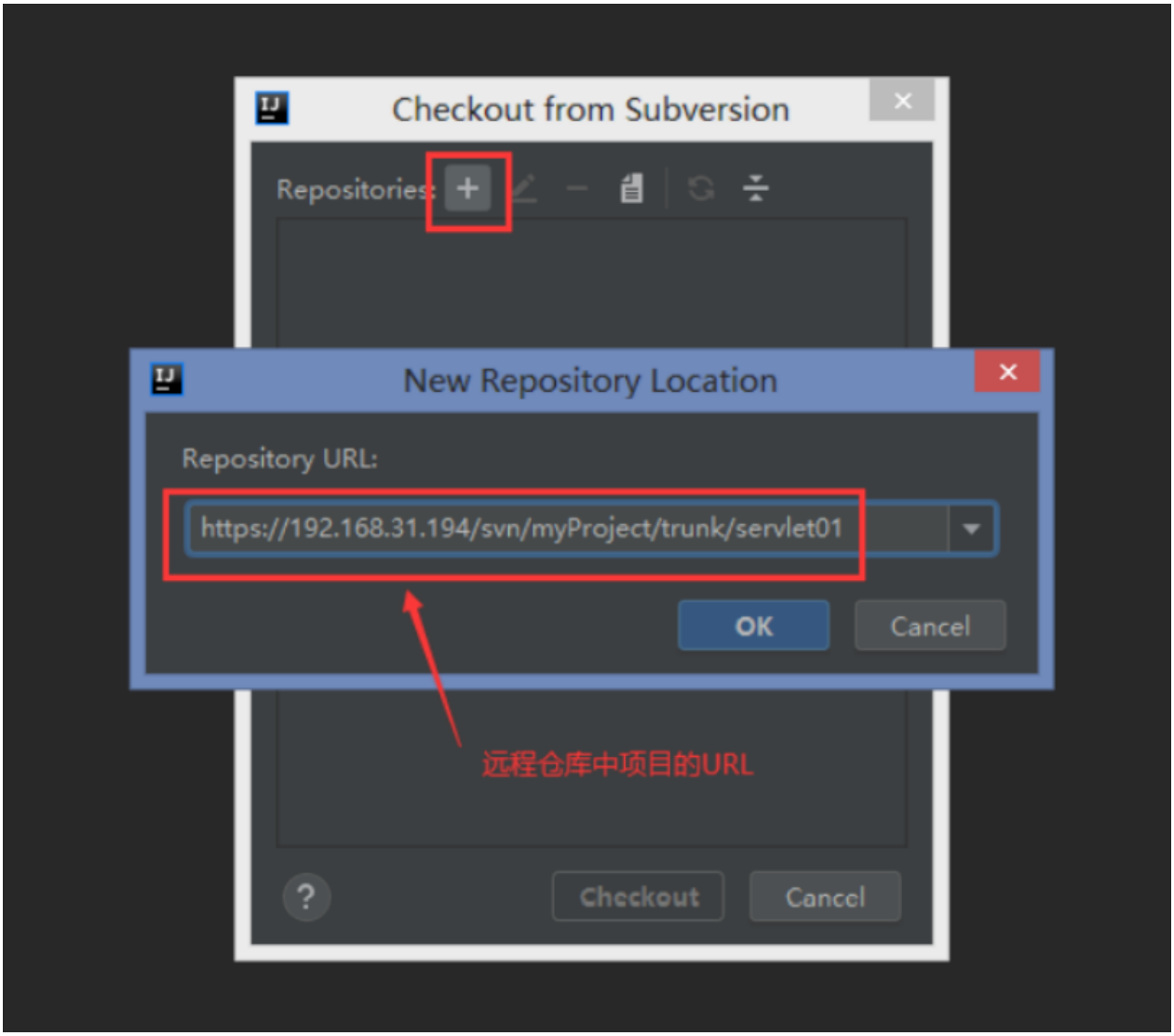
3. 重启Idea

4.2 检索项目（git中clone操作）

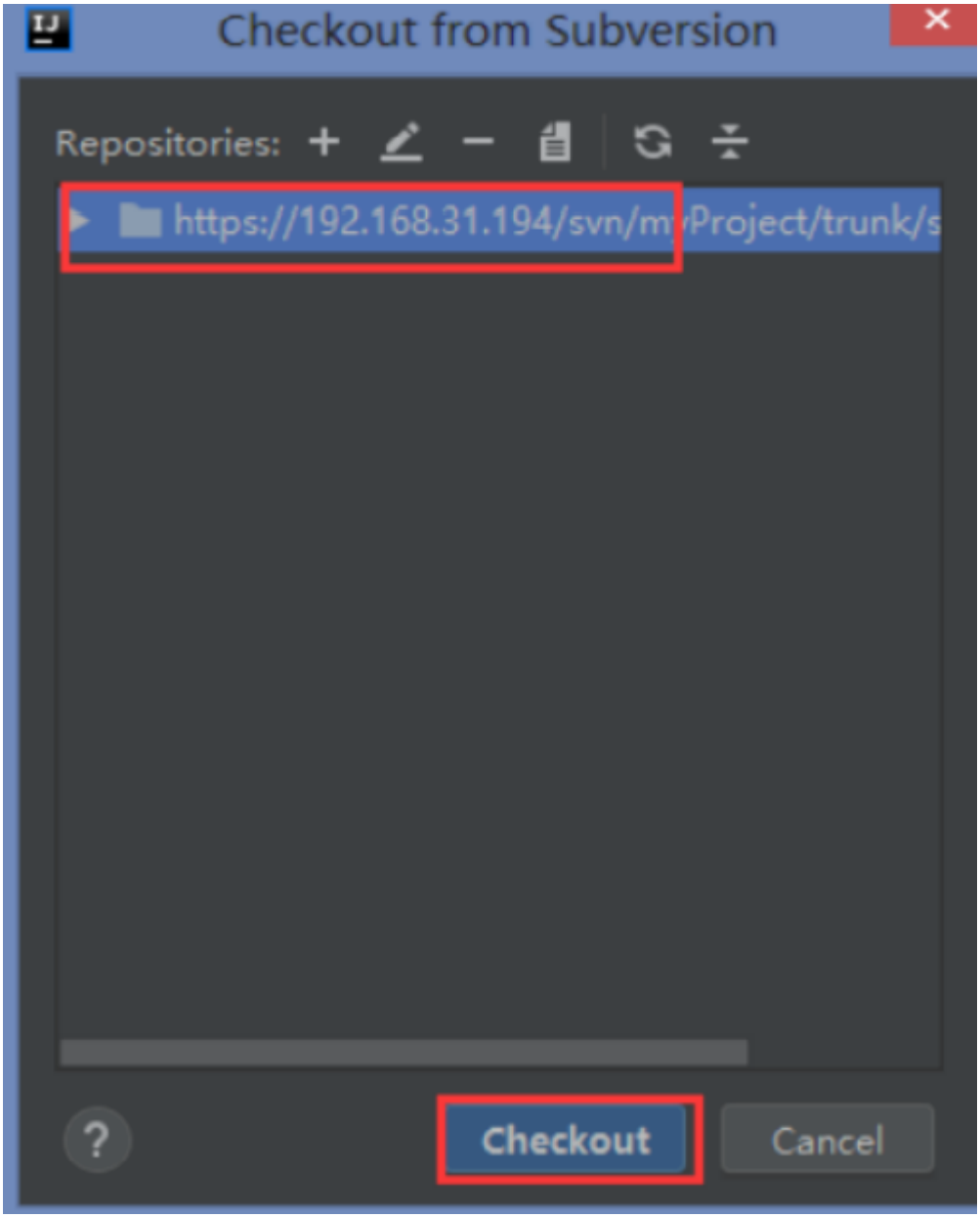
1. 选择 VCS —> Checkout from Version Control —> Subversion



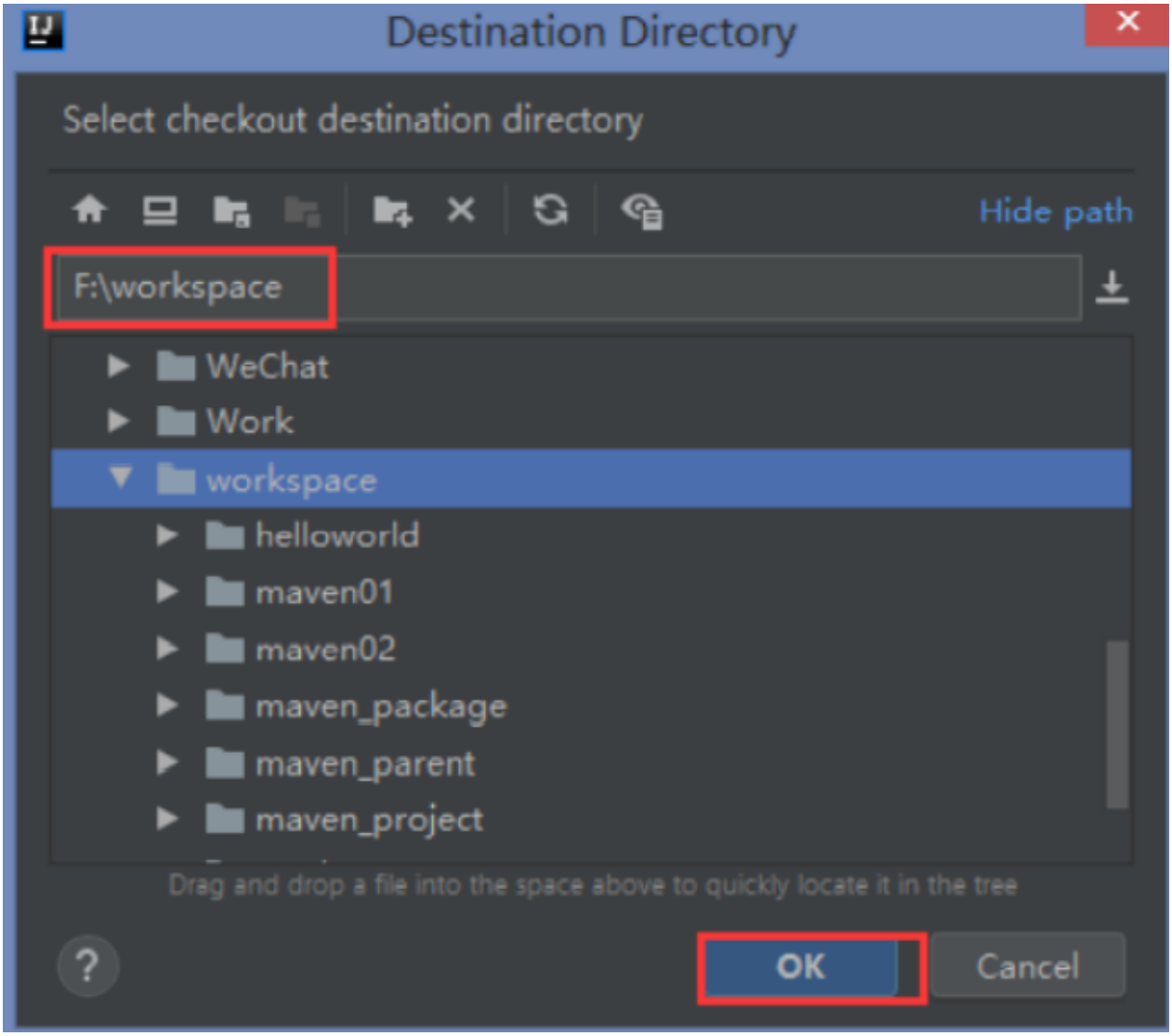
2. 添加远程仓库中项目的URL



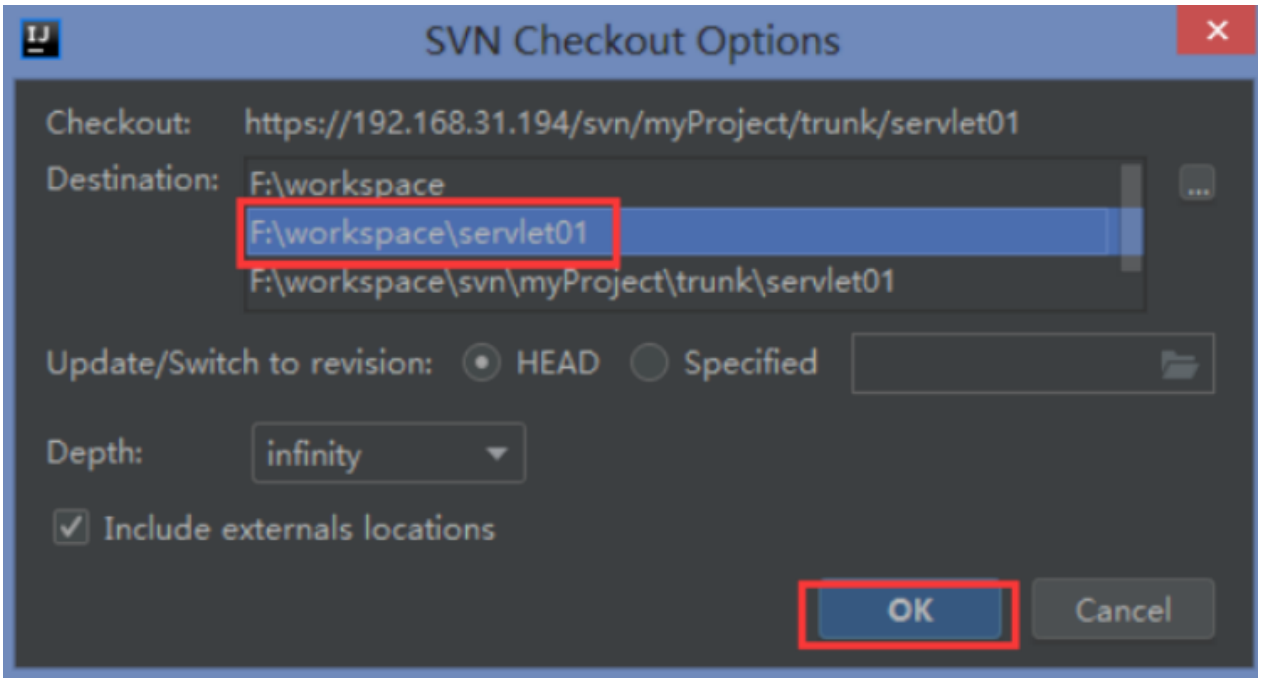
3. 点击添加的URL，选择 Checkout



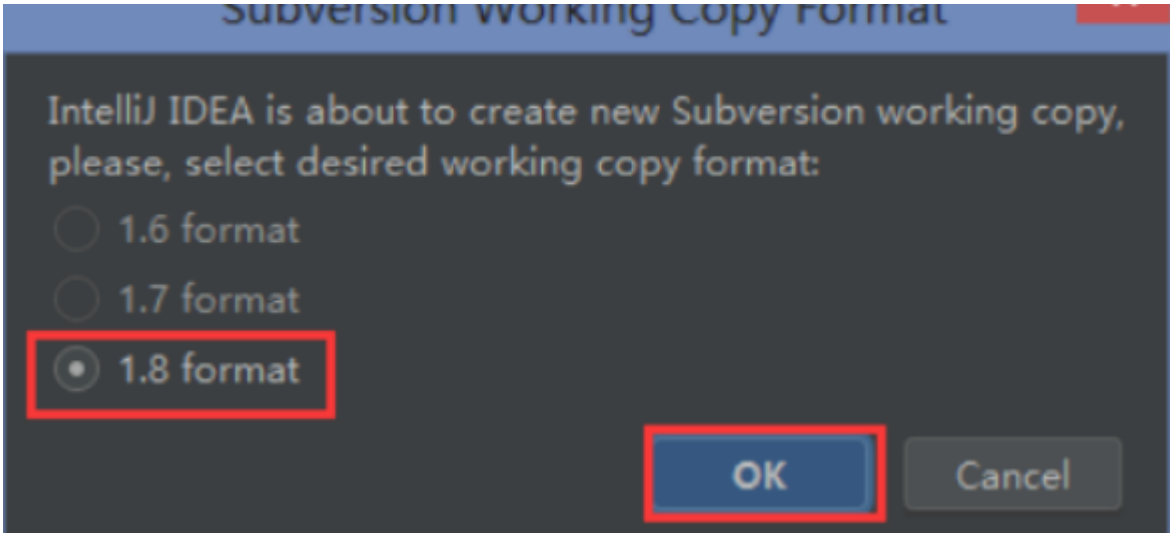
4. 选择检索的项目的存放位置



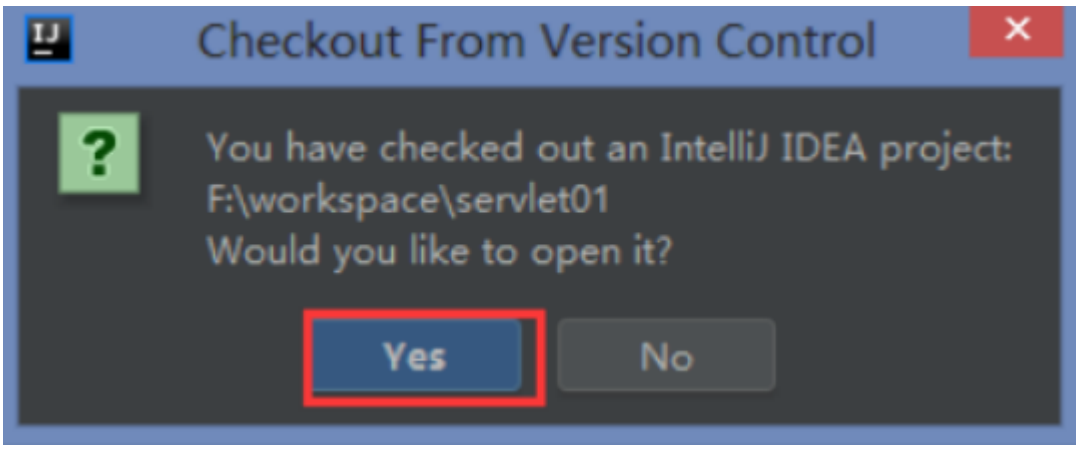
5. 选择Destination，根据自己的偏好选择，其他配置默认，单击 OK



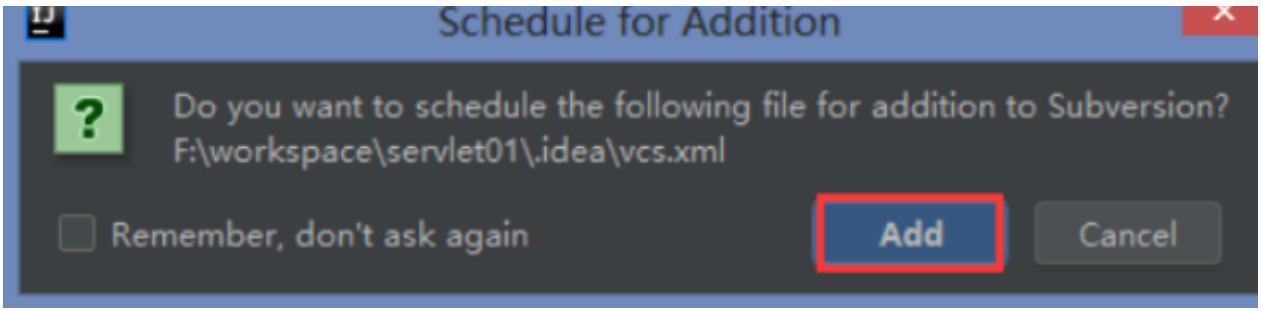
6. 选择 1.8 Format，点击 OK



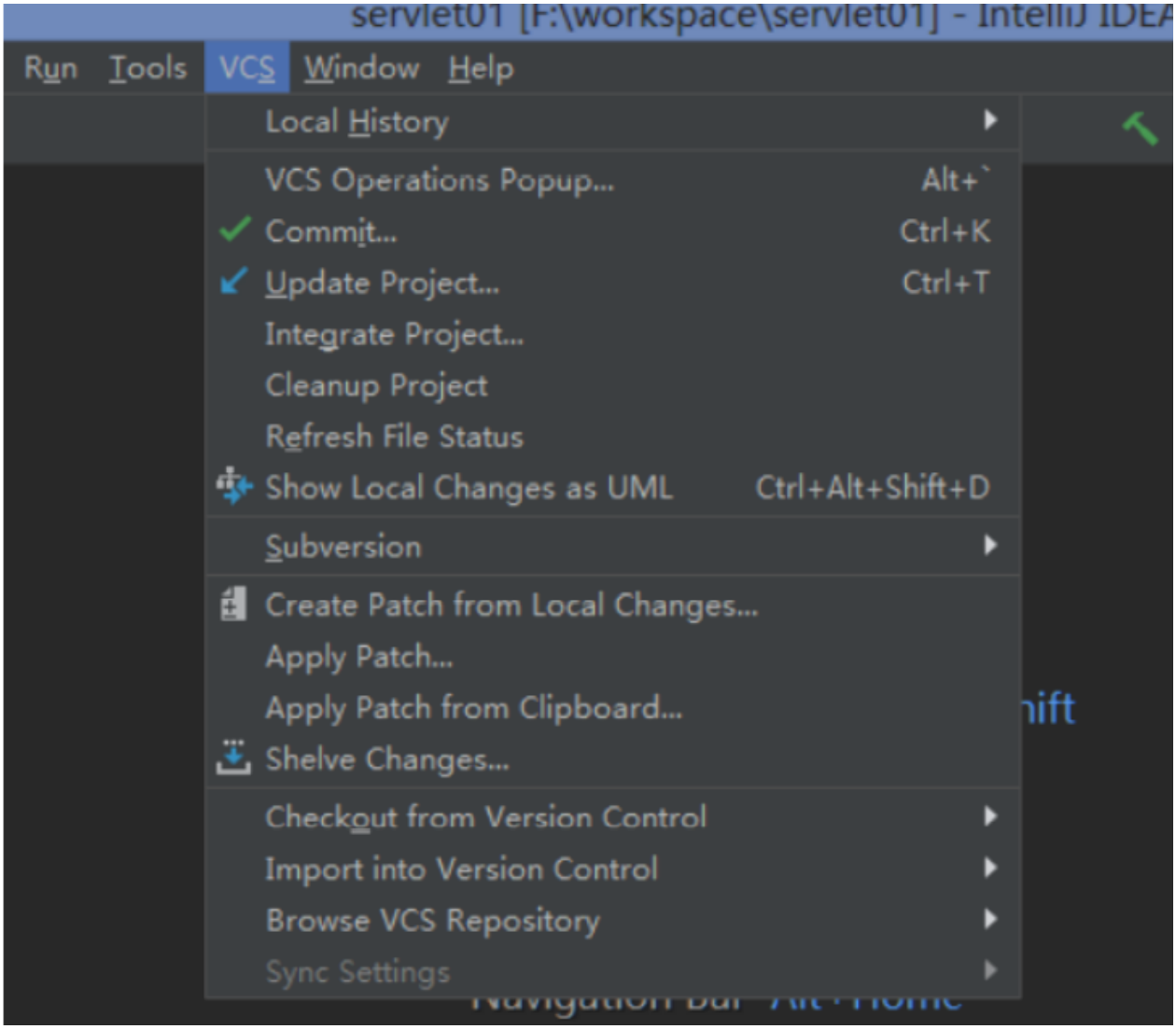
7. 已经check out一个项目，是否要打开，选择 Yes



8. 选择 Add

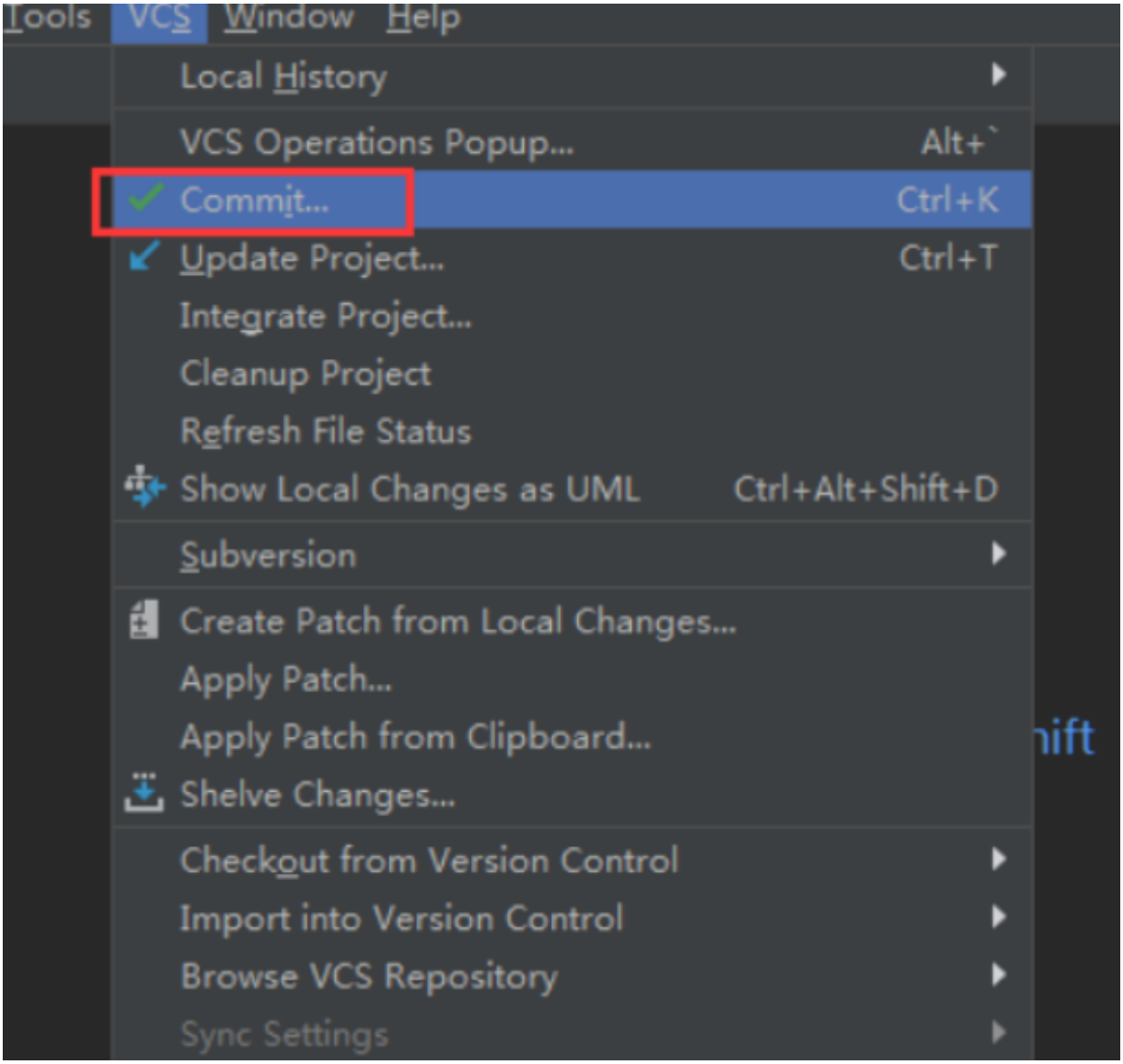


9. 此时，项目就可以与远程仓库关联

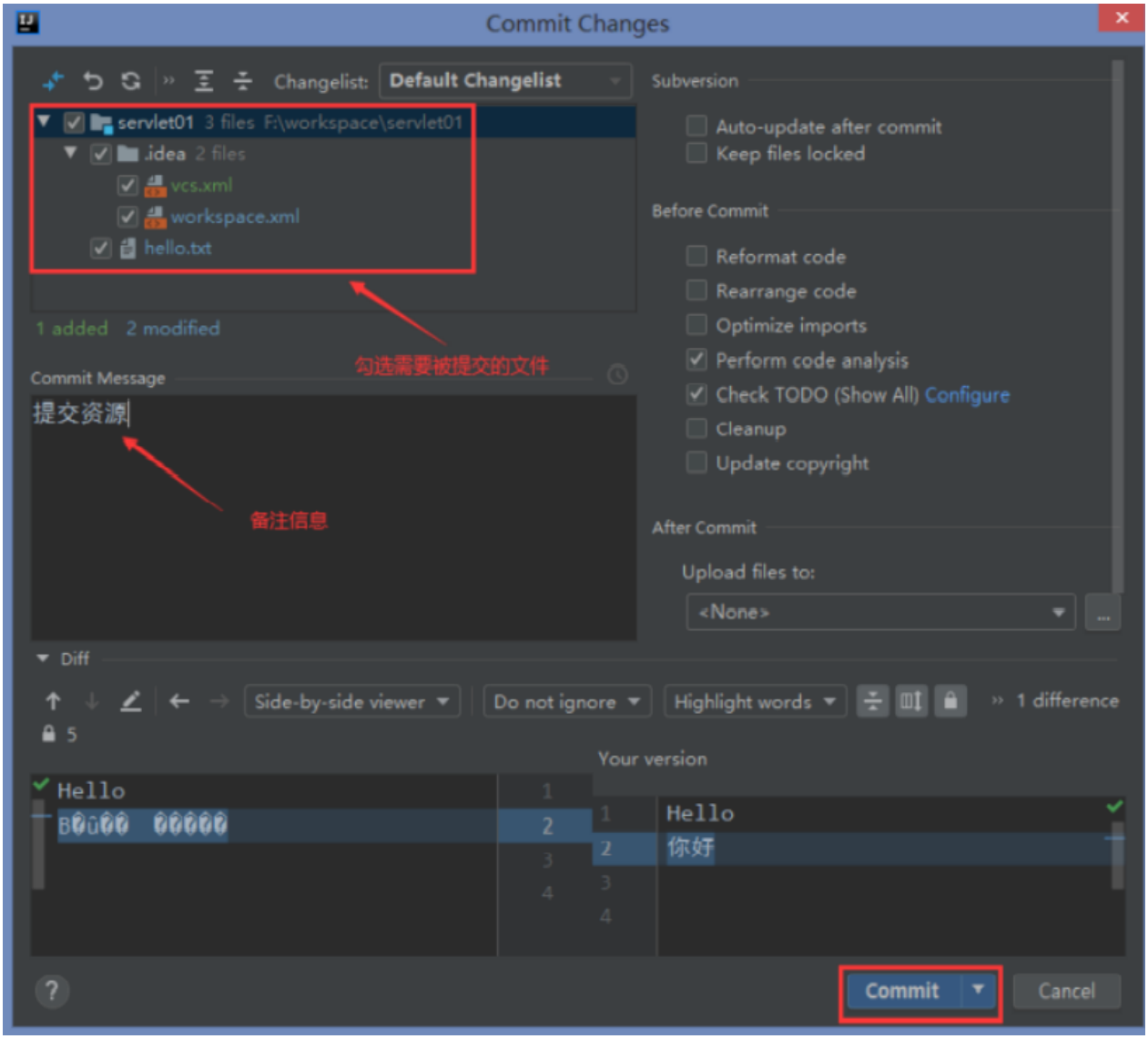


4.3 提交代码

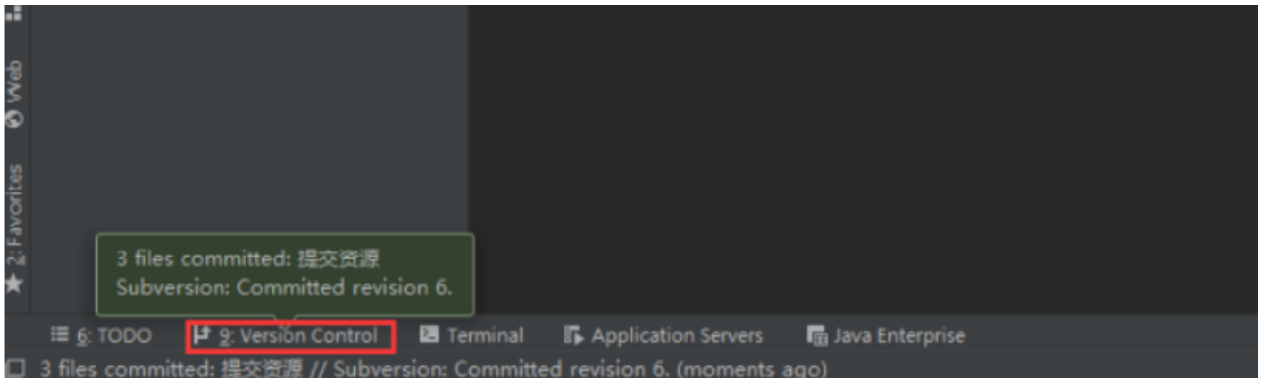
1. 选择 VCS —> Commit...



2. 选择需要提交的文件，填写提交信息，选择 Commit



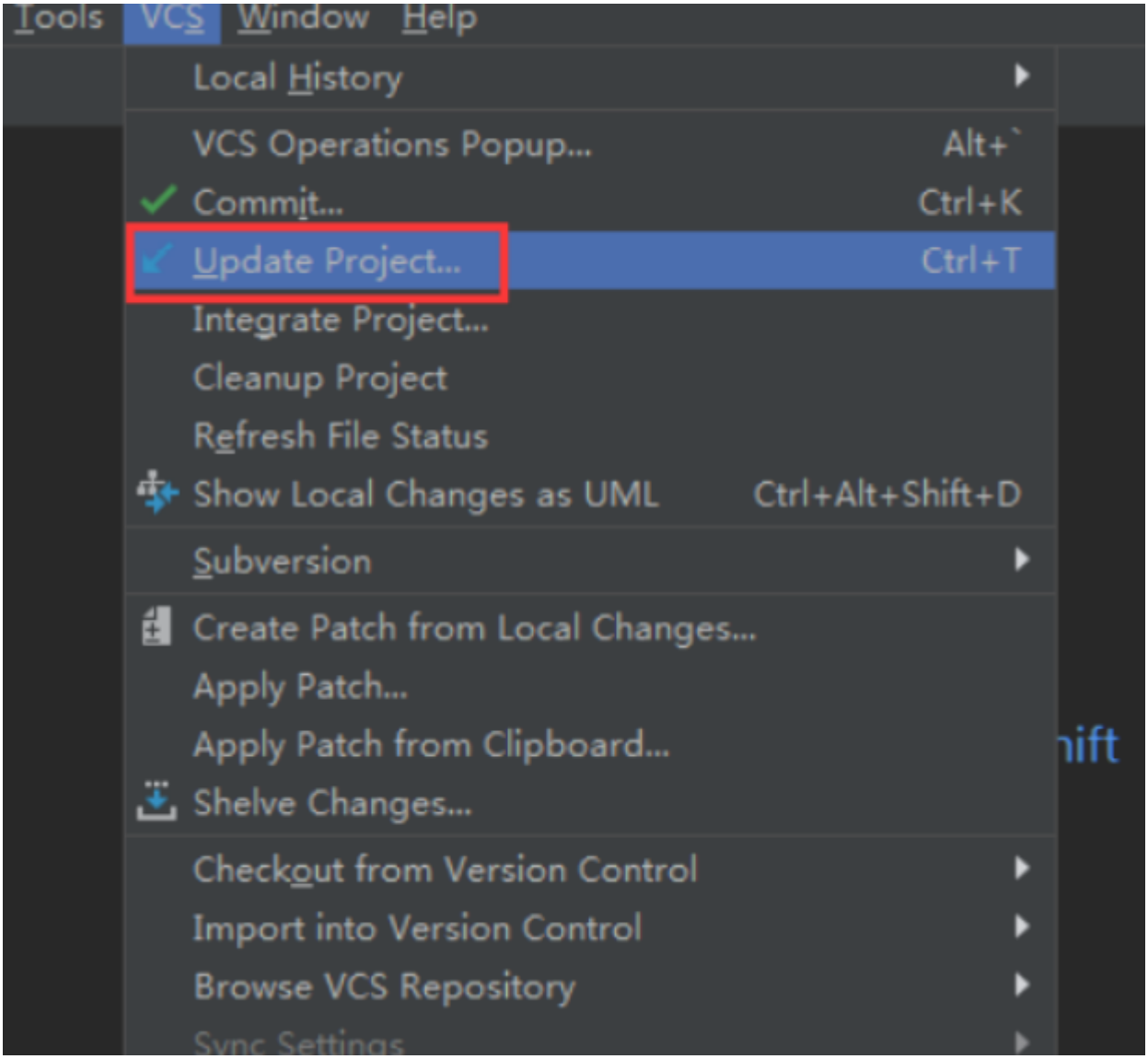
3. 提交成功后，会在Idea最下面显示提交状态



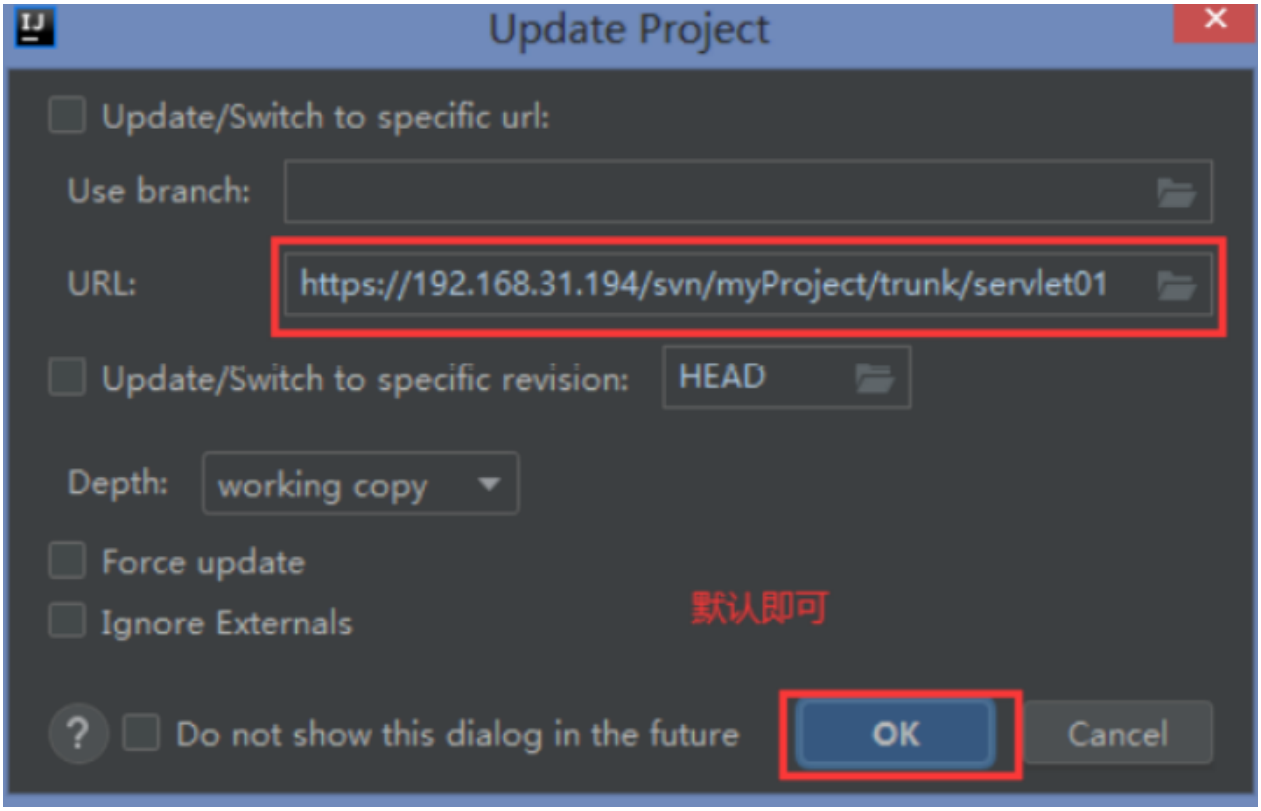
PS:提交前，先进行更新操作

4.4 更新版本

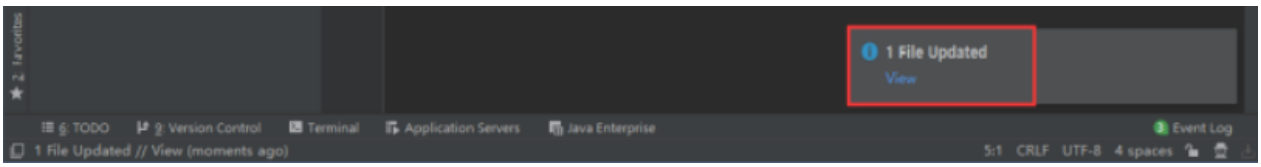
1. 选择 VCS —> Update Project...



2. 默认即可，直接选择 OK

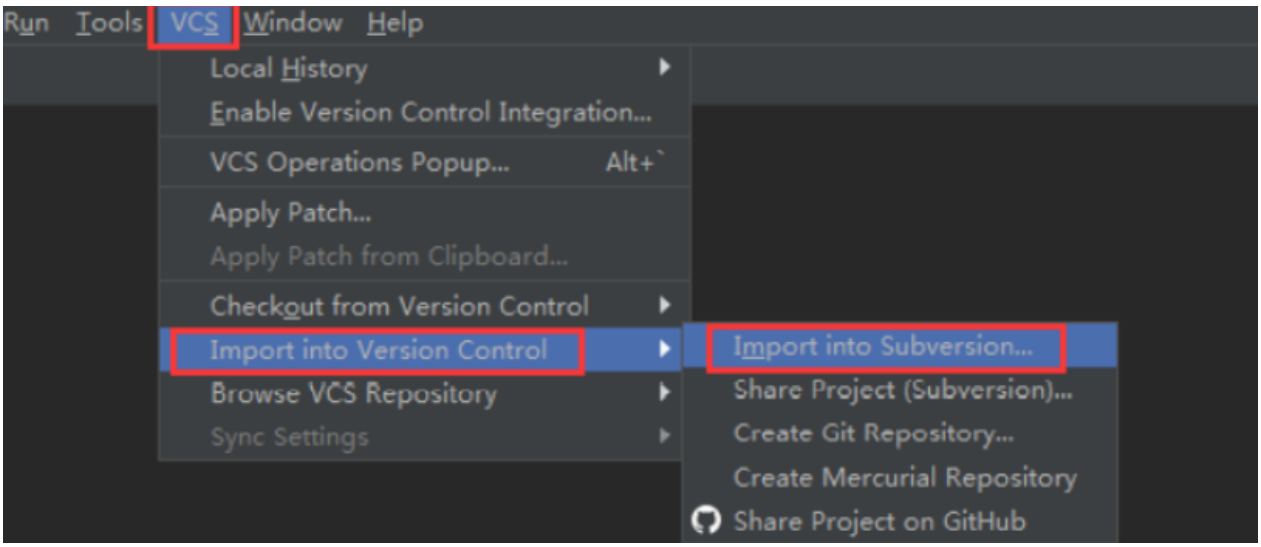


3. 更新成功的提示信息

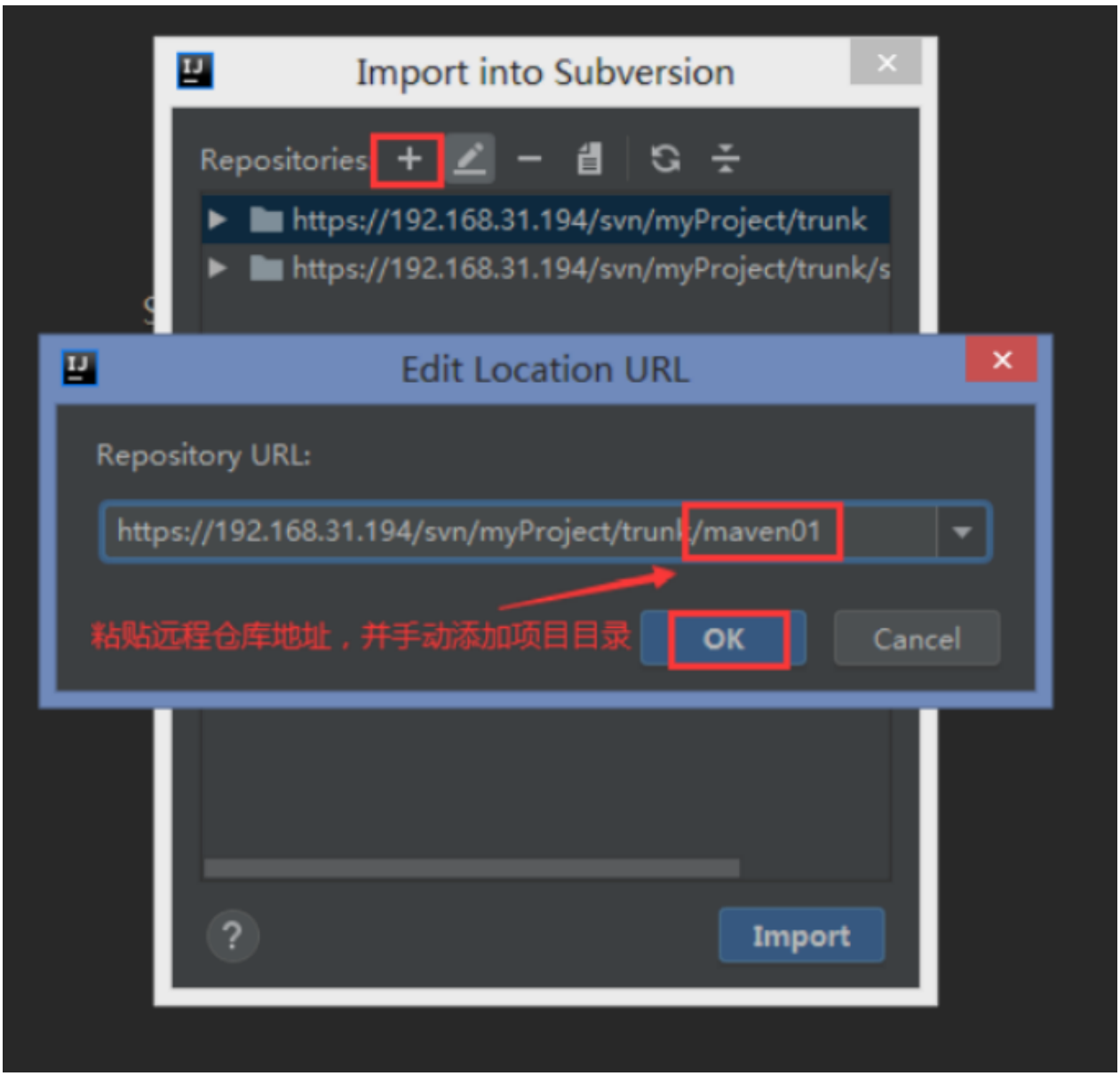


4.5 项目导入远程仓库

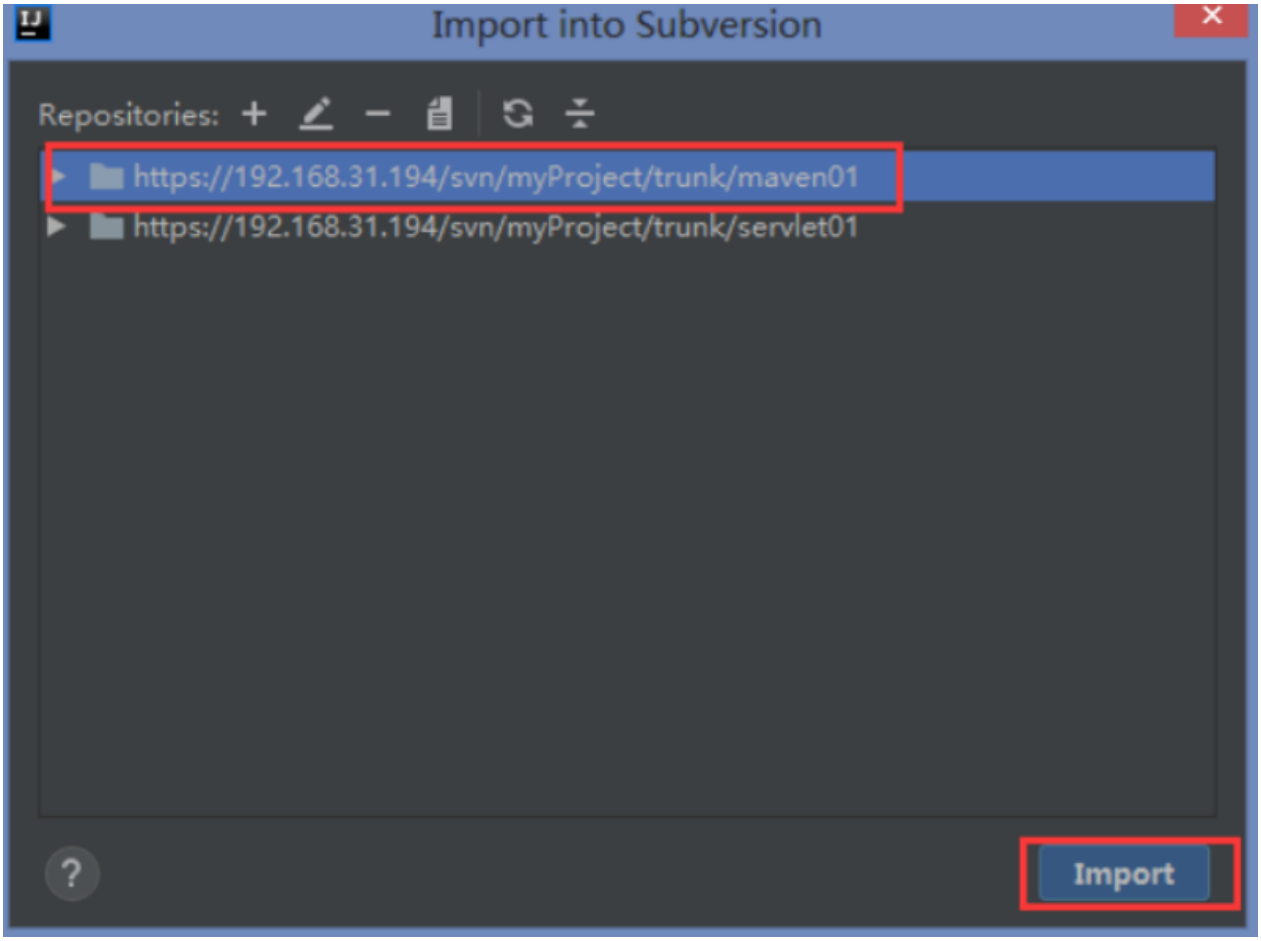
1. 选择 VCS —> Import into Version Control —> Import into Subversion



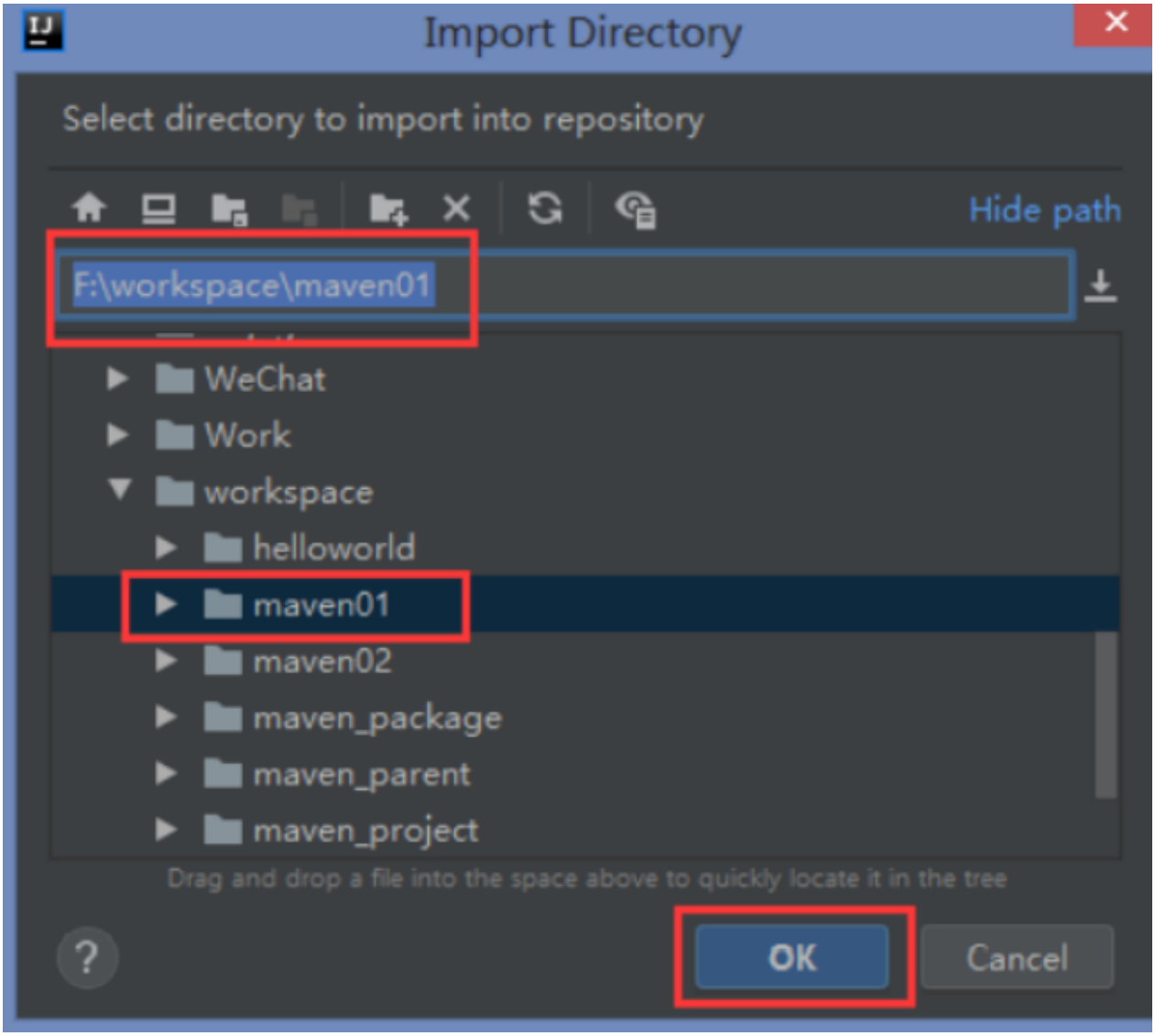
2. 选择 "+" 添加项目导入的地址（可手动添加一个文件夹，项目中的文件会放置在该文件夹中，文件名自定义）



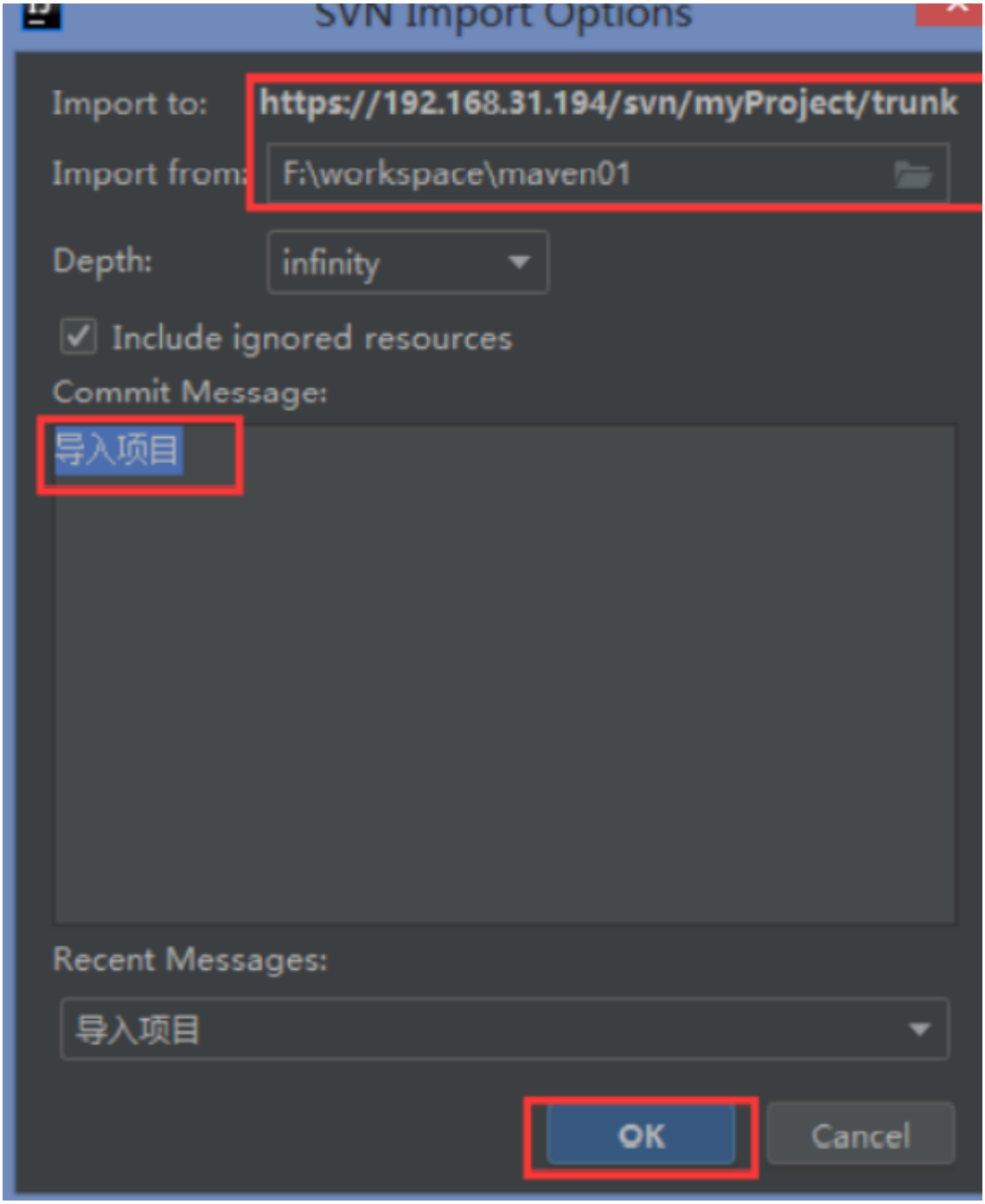
3. 选择要导入的路径，选择 Import



4. 选择要导入的项目，点击 OK



5. 检查导入的路径，填写导入信息，选择 OK

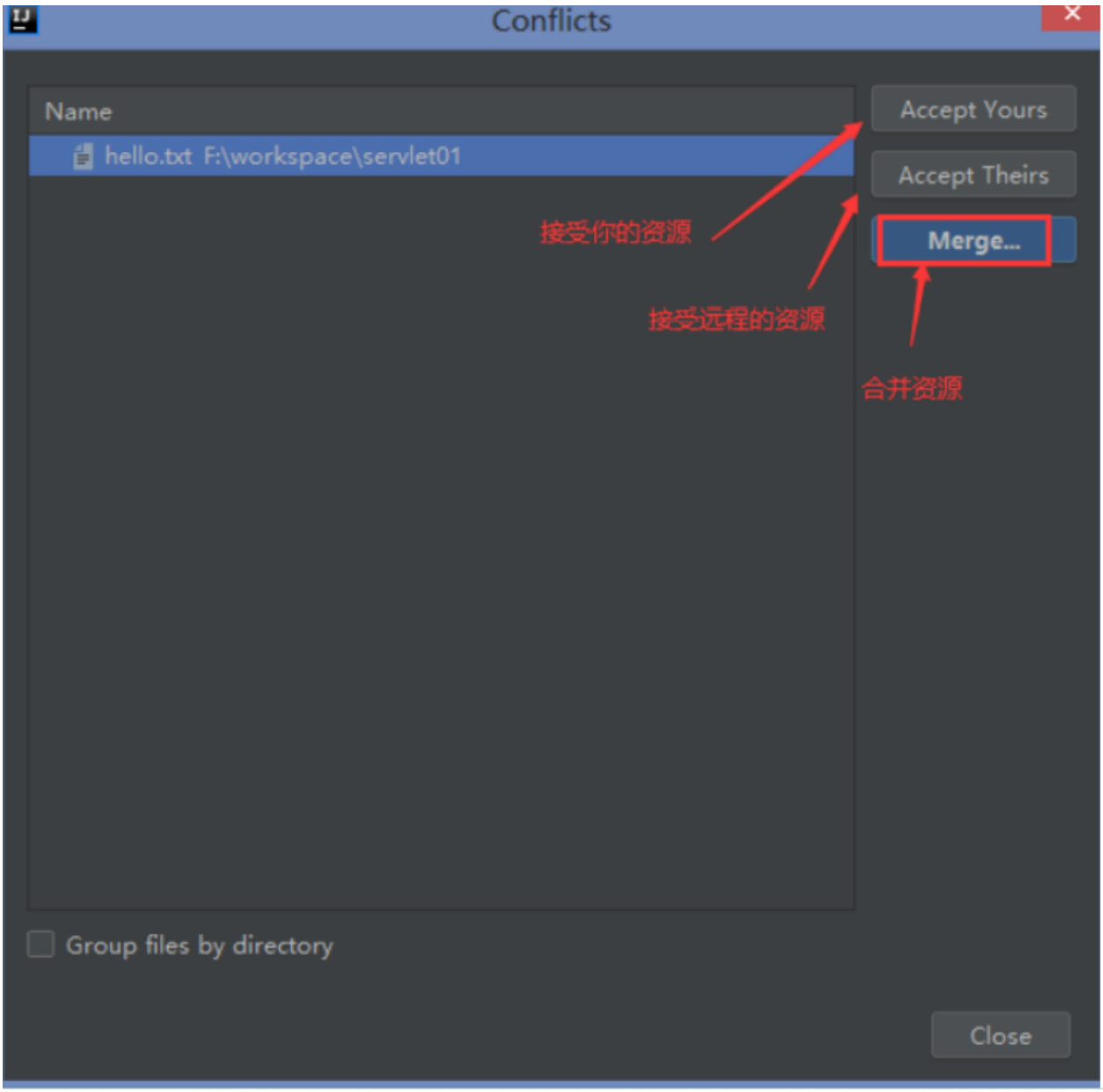


6. 在远程仓库中检查是否导入成功即可。

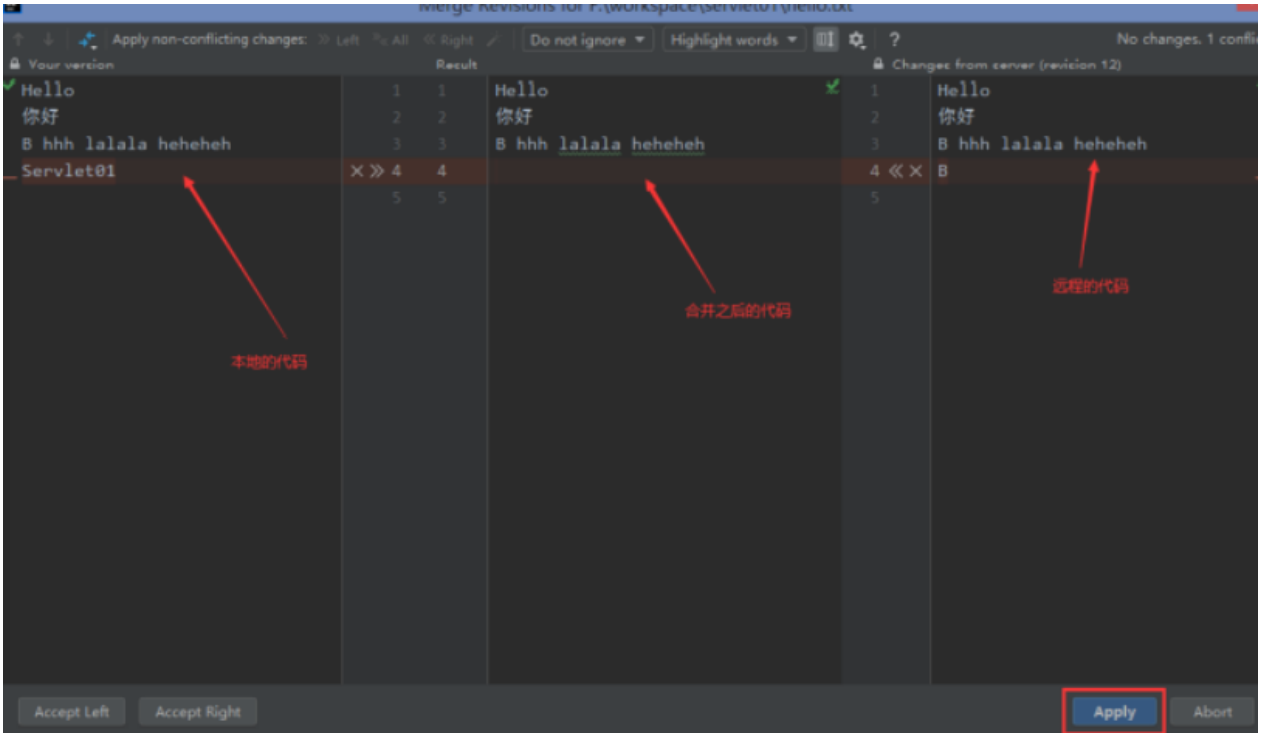
4.6 版本冲突解决

1. 如果未更新，就提交资源（有其他用户也提交过资源），则提交失败

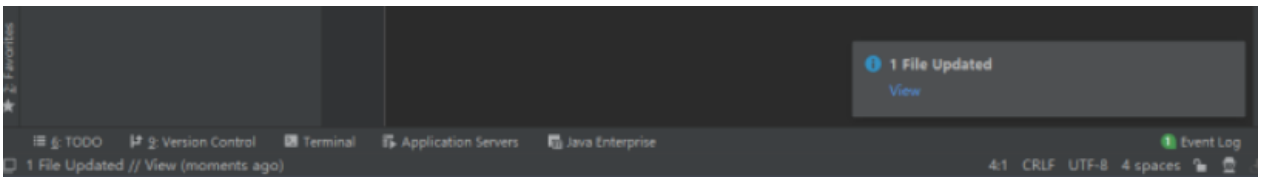
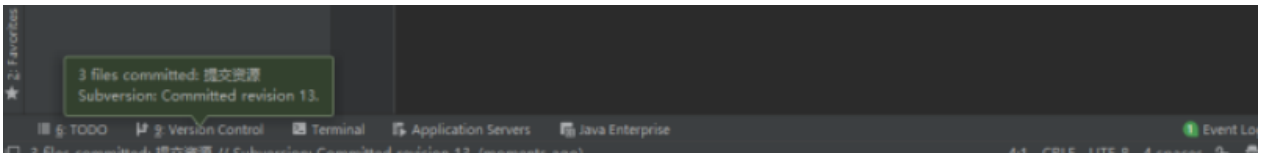
2. 此时，执行更新操作，将其他人提交过的资源更新到本地，会提示冲突



3. 通常选择合并，再选择需要保留的代码，选择好之后选择 Apply



4. 提示更新成功



5. 再次选择提交，成功解决冲突